RAMDE - Engenharia Orientada a Requisitos e Modelos

Mestrado em Engenharia de Sistemas Computacionais Críticos

Lecture 01

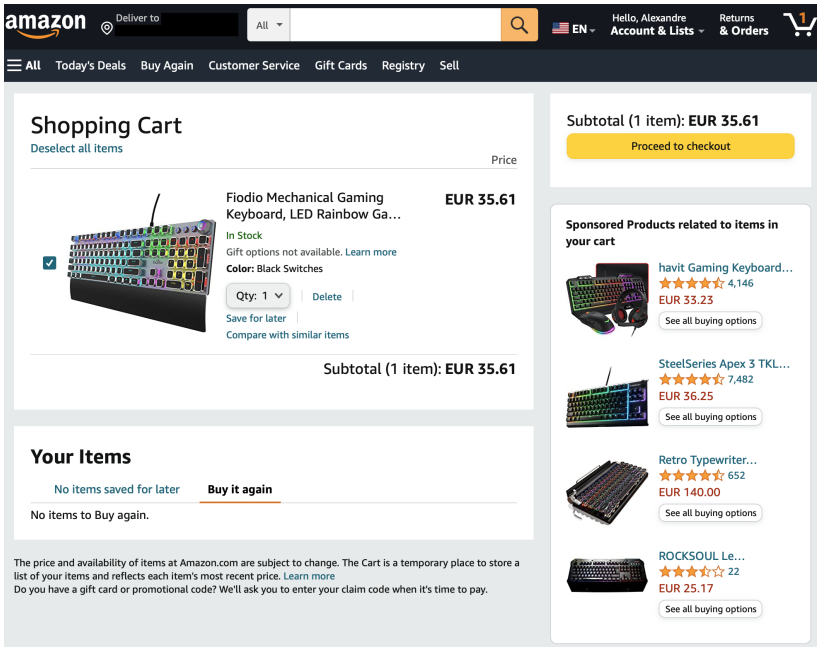
Introduction

Alexandre Bragança atb@isep.ipp.pt

Dep. de Engenharia Informática – ISEP

2026/2027

Motivation: Requirements, Models and Critical Computing Systems (1/6)



Consider the following functional requirement for an e-commerce system:

"While buying, when my basket is not empty, if I checkout, after paying, the system shall generate a receipt"

Problem: How can I verify if the system includes this functionality?

Motivation: Requirements, Models and Critical Computing Systems (2/6)

Consider the following functional requirement for an e-commerce system:

"While buying, when my basket is not empty, if I checkout, after paying, the system shall generate a receipt"

Problem: How can I verify if the system includes this functionality?

Possible/Usual Solution: Usually, I could **perform a test of the system, simulating this scenario, and check if the result is what is expected**.

However... This will only show that for the specific case (and data) tested, the system performed as expected in the requirement. There are no guarantees that the system will perform as required in all possible cases. And it is unfeasible/impossible to test all the possible scenarios!

Maybe this is acceptable, and not having a receipt is not a critical problem. We can always generate the missing receipts later...

Motivation: Requirements, Models and Critical Computing Systems (3/6)



Now consider the following functional requirement for a car:

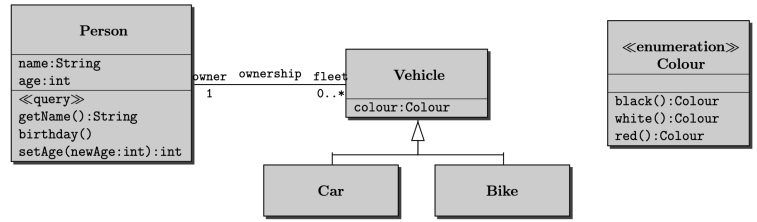
"While driving, when the driver hits the braking pedal, the braking system should be activated to start decelerating the car"

Problem: If, as previously, I only test for a specific scenario, will this be enough? **Will I consider my system reliable?...**

Maybe we need some guarantee or proof that this requirement will always stand!

Motivation: The Vision: Models... (4/6)





"All cars of a person are black":

```
context  Person
inv:     self.fleet->forAll(v | v.colour = #black)
```

Models, and modeling, allows to construct abstractions of systems with only the specific aspects required for the aim of the model. This reduces the domain of possible scenarios to test, from infinite to something that is manageable.

Models are also a formal way to specify a system and its requirements. This enables the use of mathematical constructs to verify and prove properties of the system (requirements of the system). It is no longer required to test all possible scenarios.

There are several approaches, but it is usual to use some kind of **state machine to model the behavior** of systems and some kind of **logic to model its requirements or constraints**.

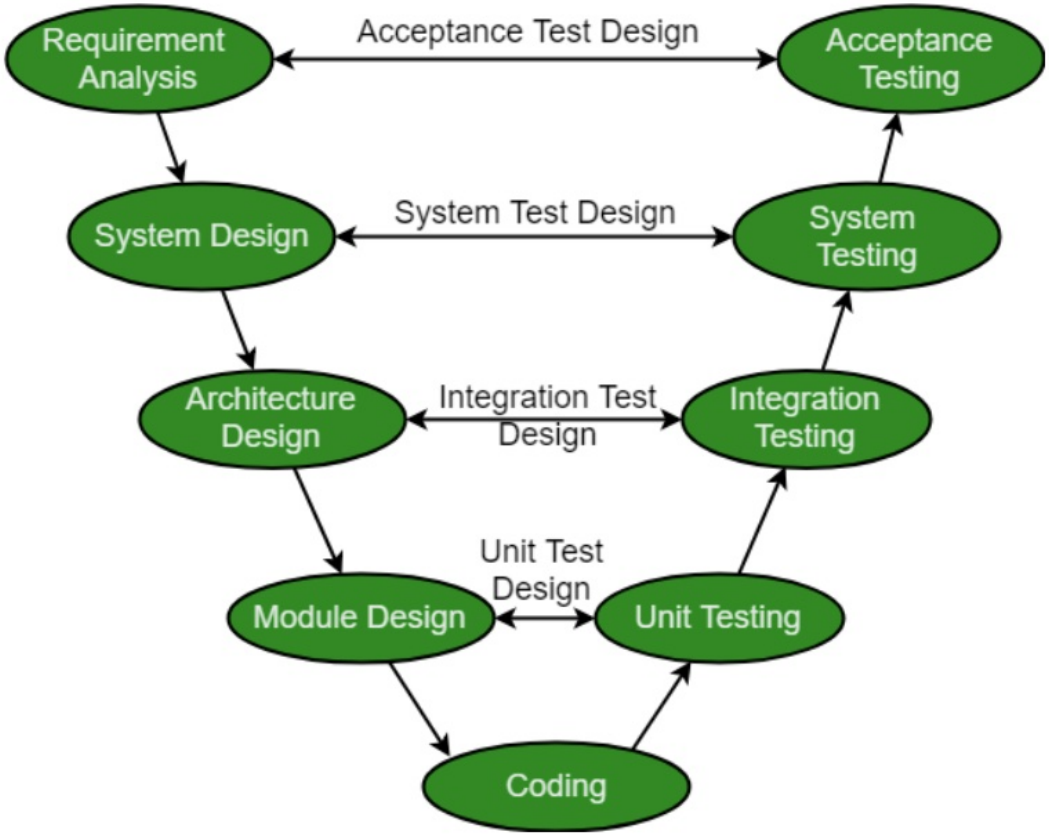
Motivation: The Vision: Limitations... (5/6)

So, that's it, **Model-Driven Engineering (MDE)** solves everything?... Not so far, it helps, **but several issues are still there**, for instance:

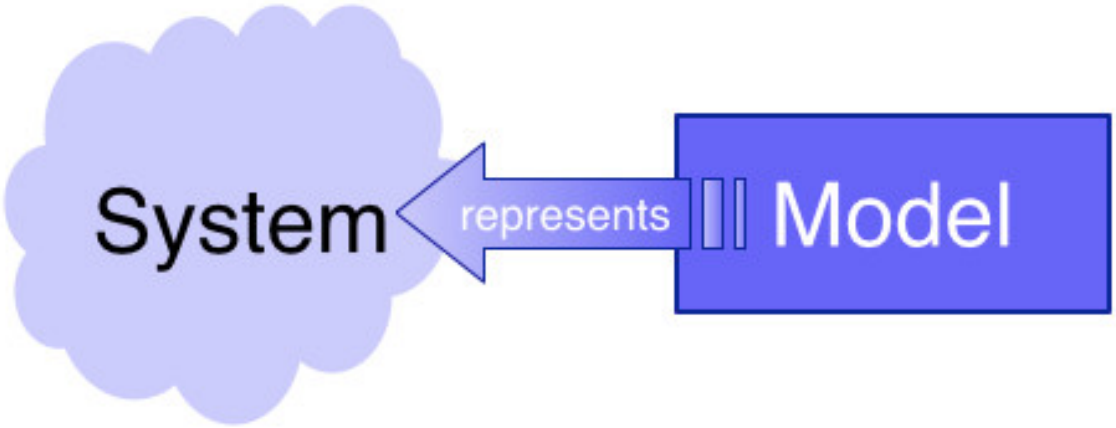
- **Sometimes proving the behavior of some abstraction model is not enough or not possible**. For instance, in the previous example, the behavior of the car can be impacted by the environment (i.e., weather, state of the tires, quality of the road). More specific models and tests may be required for building a reliable system
- **Models must be converted to code**. This can be a manual, semi-automatic, or full-automatic task. Some features of MDE, such **treating code as models**, can help in this context
- **Eliciting and specifying requirements is also a problem**, since stakeholders and software engineers may not understand each other. MDE and **Domain-Specific Languages (DSL)** can help in this context

Motivation: Use Best Practices: The SDLC V-Model (6/6)

The Software Development Life Cycle V-Model (often used in safety-critical systems), also known as Verification and Validation model, it is based on the association of a testing phase for each corresponding development stage. This (and other practices) improve software quality.

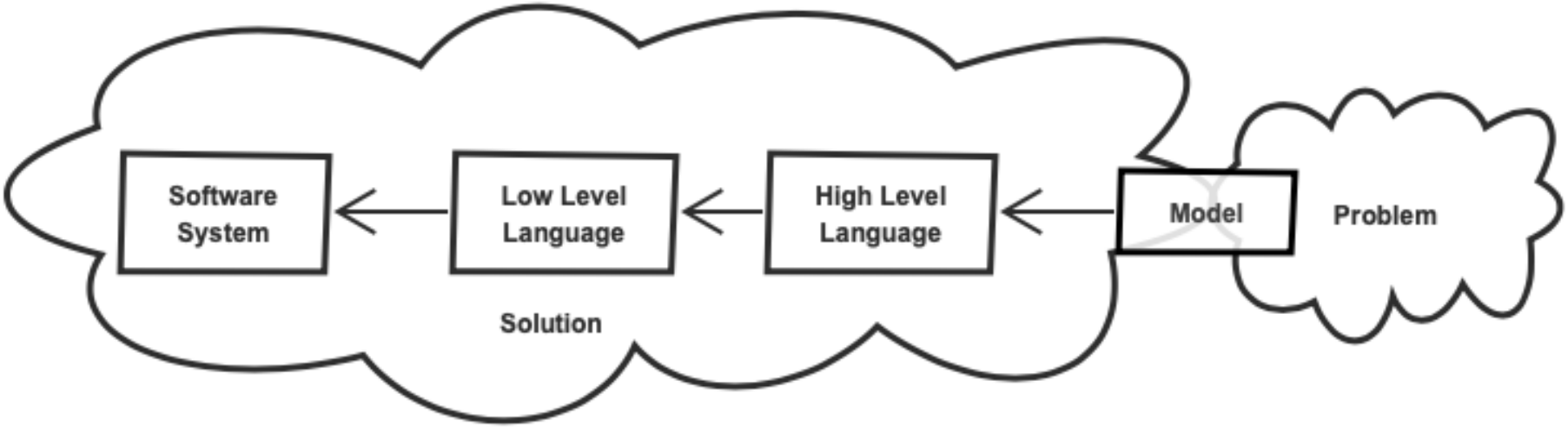


Base Concepts: What is a Model?...



- **Model**: a simplified or partial representation of reality, defined in order to accomplish a task or to reach an agreement.

Base Concepts: A Model in Model-Driven Engineering



- **Model**: a representation of a software system, using high level concepts, enabling a connection between problem and solution spaces.

Goal

This course addresses **model-driven engineering (MDE) for Critical Systems** by focusing on subjects such as

- metamodeling and domain-specific languages
- modeling standards
- requirements elicitation, specification and analysis
- verification, validation, and testing

Outcomes

By successfully passing this course, the student must be able to understand the principles of MDE. Specifically, by the end of the course, the student must be able to:

- **Understand the fundamental concepts and techniques of MDE**, its role in a critical system's development and safety assessment processes, and apply the acquired knowledge in practical use-cases;
- **Identify and model functional and non-functional requirements for critical systems**, using the best practices and ensuring that they respect the expected quality standards (verifiable, traceable, unambiguous, correct, etc.);
- **Understand the software development process**, in particular requirements engineering and software validation and verification activities, **demonstrating the ability to apply the process in practical cases**;
- **Acquire hands-on experience by using tools** that support requirement specification and management, system modelling and evaluatin, verification, model checking and tests.

Contents

Contents

- Fundamental concepts, principles and techniques of MDE, and its specificities when applied to critical systems
- Fundamentals of requirement engineering
- UML and SysML as software and system modeling languages for enabling critical systems MDE
- Formal specification and verification of critical systems models: principles and tools.
- The software development process in the MDE context and for critical systems
- Fundamentals of testing and fault analysis, and its usage in MDE
- Case studies

Bibliography and Documents

- **Lecture Handouts** (pdf)
- Marco Brambilla, Jordi Cabot, and Manuel Wimmer. "**Model-Driven Software Engineering in Practice: Second Edition (2nd. ed.)**". Morgan & Claypool Publishers, 2017.
- Pierre Bourque, Richard E. Fairley, and IEEE Computer Society. "**Guide to the Software Engineering Body of Knowledge (SWEBOK(R)): Version 3.0 (3rd. ed.)**". IEEE Computer Society Press, Washington, DC, USA, 2014
- Sanford Friedenthal, Alan Moore, Rick Steiner, "**A Practical Guide to SysML: The Systems Modeling Language, 2nd Edition**", Elsevier, 2015
- Alexander Kossiakoff, William N. Sweet, Samuel J. Seymour, Steven M. Biemer. "**Systems Engineering Principles and Practice 3rd Edition**", Wiley, 2022
- Klaus Pohl, "**Requirements Engineering: Fundamentals, Principles, and Techniques**", 2010th Edition, Springer
- Christel Baier and Joost-Pieter Katoen, "**Principles of Model Checking**", MIT Press, 2008

Teaching/Learning Tools

- Moodle (`https://moodle.isep.ipp.pt`)
- Git (`https://git-scm.com`)
- GitHub (`https://github.com`)
- Eclipse (`http://www.eclipse.org`)
  - Eclipse SysML plugin (Papyrus - `https://www.eclipse.org/papyrus/`)
- Others, as indicated by teacher during classes

Previous knowledge assumed as acquired

- Knowledge and **experience in programming** (e.g., in the java language (LEI: APROG, PPROG, LPROG).
- Software engineering knowledge and experience, mainly **analysis and design best practices with object oriented paradigm and UML** (LEI: ESOFT, EAPLI).
- Basic knowledge and **experience with version control systems** (LEI: LAPR2, LAPR3, LAPR4 and LAPR5).

Classes - Lectures

Lectures

- Lecture classes will introduce and motivate the relevance of MDE in the development process of any critical system.
- In these classes, the theoretical foundations, principles and techniques associated with the process of modeling a system using an MDE approach will be introduced. Special focus will be given to using models for requirements specification
- Model verification and testing will also be addressed in terms of their principles and explored with concrete examples and case studies.
- In each of the topics addressed in these lectures, practical examples and exercises will be given to students

Classes - Lab

Lab

- Lab classes will be dedicated to complement necessary extra technical content of the lecture classes that is fundamental to enable students **to use appropriate software tools to validate the knowledge they acquired in practical MDE exercises**
- During these classes, there will also be time allocated for the lecturer **to mentor the class project being developed individually**
- This mentoring will focus on providing early feedback to help the students mitigate possible flaws on their models, guiding them on the maintenance of requirements, consistency of the models, and defining verification and testing efforts
- **The project, being iterative and incremental, will contribute to all learning outcomes**, since it will lead the students to bridging the gap between what they learn in the lecture classes and the software skill being acquired in the lab classes, in a continuous and goal oriented way

Evaluation

Evaluation during the semester (project, 100%) - no final written exam.

- Students will work in an **individual project** during the semester
- Each student will work within a specific GitHub repository
- **Each student should propose a working topic for the project** (something related to critical systems, like home automation system or automatic warehouse systems)
- All artifacts produced during the project (including documentation) are required to be committed to the repository.
- **Students who are free from attending classes must develop the project outside classes**. They must comply with all the other rules (e.g., deadlines).
- According to the model defined for the master's degree, **the use of AI tools is permitted to support the learning process**, with a statement about the tools used and their respective purposes, and student accountability for the decisions made and content presented.

Evaluation - Project

Project [100%]

Project to be developed during the semester and divided into 3 evaluation moments

- **M1**: Evaluation during a Lab class, Weeks 3 to 10 (40%)
  - The work during the lab classes
  - A Session in which the teacher monitorizes the student work in the ongoing project
- **M2a**: Final Project Evaluation, Week 11, (40%)
  - Final defense of the project, from a practical perspective
- **M2b**: Final Project Evaluation, Week 11, (20%)
  - Final defense of the project, from a theoretical perspective

Evaluation Rubric

A performance scale will be available to the students so that they can be aware of their own performance (and it will be used in the evaluation/assessment of each component of each part of the project):

- **0- nothing**. No submission or irrelevant.
- **1- tentative (4/20)**. Do not achieve any of the requirements
- **2- developing/insufficient (8/20)**. Achieves only a subset of the requirements
- **3- acceptable (12/20)**. Achieves the requirements but with major faults
- **4- good (16/20)**. Achieves the requirements with minor faults; justifies options
- **4,5- very good (18/20)**. Achieves the requirements with minor faults; justifies options, analyzes alternatives in a qualitative way
- **5- excellent (20/20)**. Achieves completely the requirements; justifies options; analyzes alternatives in a quantified and rigorous way (e.g., implementing alternative(s))

Tools

For source control

- **Git** (`https://git-scm.com`)

For repository hosting

- **GitHub** (`https://github.com/`)
- **Important: Please register your account using your official ISEP email and make it public in your Github profile!**

For modeling

- Install Eclipse (http://www.eclipse.org)
  - Please **use the 2026-06 version of Eclipse**
  - Download the Package "Eclipse Modeling Tools"
  - After installation, run Eclipse and select "Help/Install New Software", and then select to work with the url of your Eclipse version. Then, Select "Modeling" and then select and install all components
- Install the following Eclipse additional plugins:
  - Plantuml (see http://plantuml.com/eclipse)